

every node has a path to s . Then s and v are mutually reachable for every v , and so it follows that *every* two nodes u and v are mutually reachable: s and u are mutually reachable, and s and v are mutually reachable, so by (3.16) we also have that u and v are mutually reachable.

By analogy with connected components in an undirected graph, we can define the *strong component* containing a node s in a directed graph to be the set of all v such that s and v are mutually reachable. If one thinks about it, the algorithm in the previous paragraph is really computing the strong component containing s : we run BFS starting from s both in G and in G^{rev} ; the set of nodes reached by *both* searches is the set of nodes with paths to *and* from s , and hence this set is the strong component containing s .

There are further similarities between the notion of connected components in undirected graphs and strong components in directed graphs. Recall that connected components naturally partitioned the graph, since any two were either identical or disjoint. Strong components have this property as well, and for essentially the same reason, based on (3.16).

(3.17) *For any two nodes s and t in a directed graph, their strong components are either identical or disjoint.*

Proof. Consider any two nodes s and t that are mutually reachable; we claim that the strong components containing s and t are identical. Indeed, for any node v , if s and v are mutually reachable, then by (3.16), t and v are mutually reachable as well. Similarly, if t and v are mutually reachable, then again by (3.16), s and v are mutually reachable.

On the other hand, if s and t are not mutually reachable, then there cannot be a node v that is in the strong component of each. For if there were such a node v , then s and v would be mutually reachable, and v and t would be mutually reachable, so from (3.16) it would follow that s and t were mutually reachable. ■

In fact, although we will not discuss the details of this here, with more work it is possible to compute the strong components for all nodes in a total time of $O(m + n)$.

3.6 Directed Acyclic Graphs and Topological Ordering

If an undirected graph has no cycles, then it has an extremely simple structure: each of its connected components is a tree. But it is possible for a directed graph to have no (directed) cycles and still have a very rich structure. For example, such graphs can have a large number of edges: if we start with the node

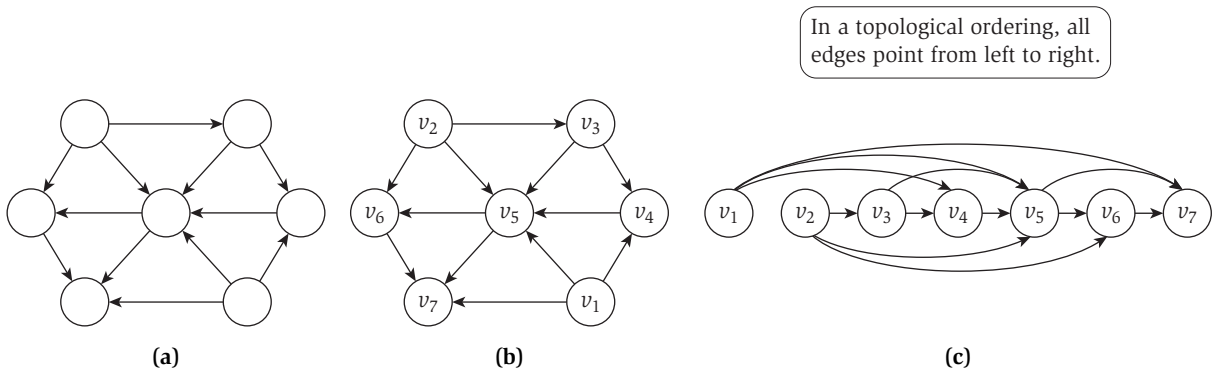


Figure 3.7 (a) A directed acyclic graph. (b) The same DAG with a topological ordering, specified by the labels on each node. (c) A different drawing of the same DAG, arranged so as to emphasize the topological ordering.

set $\{1, 2, \dots, n\}$ and include an edge (i, j) whenever $i < j$, then the resulting directed graph has $\binom{n}{2}$ edges but no cycles.

If a directed graph has no cycles, we call it—naturally enough—a *directed acyclic graph*, or a *DAG* for short. (The term *DAG* is typically pronounced as a word, not spelled out as an acronym.) In Figure 3.7(a) we see an example of a DAG, although it may take some checking to convince oneself that it really has no directed cycles.

The Problem

DAGs are a very common structure in computer science, because many kinds of dependency networks of the type we discussed in Section 3.1 are acyclic. Thus DAGs can be used to encode *precedence relations* or *dependencies* in a natural way. Suppose we have a set of tasks labeled $\{1, 2, \dots, n\}$ that need to be performed, and there are dependencies among them stipulating, for certain pairs i and j , that i must be performed before j . For example, the tasks may be courses, with prerequisite requirements stating that certain courses must be taken before others. Or the tasks may correspond to a pipeline of computing jobs, with assertions that the output of job i is used in determining the input to job j , and hence job i must be done before job j .

We can represent such an interdependent set of tasks by introducing a node for each task, and a directed edge (i, j) whenever i must be done before j . If the precedence relation is to be at all meaningful, the resulting graph G must be a DAG. Indeed, if it contained a cycle C , there would be no way to do any of the tasks in C : since each task in C cannot begin until some other one completes, no task in C could ever be done, since none could be done first.

Let's continue a little further with this picture of DAGs as precedence relations. Given a set of tasks with dependencies, it would be natural to seek a valid order in which the tasks could be performed, so that all dependencies are respected. Specifically, for a directed graph G , we say that a *topological ordering* of G is an ordering of its nodes as $v_1, v_2, \dots, v_n$ so that for every edge (v_i, v_j) , we have $i < j$. In other words, all edges point “forward” in the ordering. A topological ordering on tasks provides an order in which they can be safely performed; when we come to the task v_j , all the tasks that are required to precede it have already been done. In Figure 3.7(b) we've labeled the nodes of the DAG from part (a) with a topological ordering; note that each edge indeed goes from a lower-indexed node to a higher-indexed node.

In fact, we can view a topological ordering of G as providing an immediate “proof” that G has no cycles, via the following.

(3.18) *If G has a topological ordering, then G is a DAG.*

Proof. Suppose, by way of contradiction, that G has a topological ordering $v_1, v_2, \dots, v_n$, and also has a cycle C . Let v_i be the lowest-indexed node on C , and let v_j be the node on C just before v_i —thus (v_j, v_i) is an edge. But by our choice of i , we have $j > i$, which contradicts the assumption that $v_1, v_2, \dots, v_n$ was a topological ordering. ■

The proof of acyclicity that a topological ordering provides can be very useful, even visually. In Figure 3.7(c), we have drawn the same graph as in (a) and (b), but with the nodes laid out in the topological ordering. It is immediately clear that the graph in (c) is a DAG since each edge goes from left to right.

Computing a Topological Ordering The main question we consider here is the converse of (3.18): Does every DAG have a topological ordering, and if so, how do we find one efficiently? A method to do this for every DAG would be very useful: it would show that for any precedence relation on a set of tasks without cycles, there is an efficiently computable order in which to perform the tasks.



Designing and Analyzing the Algorithm

In fact, the converse of (3.18) does hold, and we establish this via an efficient algorithm to compute a topological ordering. The key to this lies in finding a way to get started: which node do we put at the beginning of the topological ordering? Such a node v_1 would need to have no incoming edges, since any such incoming edge would violate the defining property of the topological

ordering, that all edges point forward. Thus, we need to prove the following fact.

(3.19) *In every DAG G , there is a node v with no incoming edges.*

Proof. Let G be a directed graph in which every node has at least one incoming edge. We show how to find a cycle in G ; this will prove the claim. We pick any node v , and begin following edges backward from v : since v has at least one incoming edge (u, v) , we can walk backward to u ; then, since u has at least one incoming edge (x, u) , we can walk backward to x ; and so on. We can continue this process indefinitely, since every node we encounter has an incoming edge. But after $n + 1$ steps, we will have visited some node w twice. If we let C denote the sequence of nodes encountered between successive visits to w , then clearly C forms a cycle. ■

In fact, the existence of such a node v is all we need to produce a topological ordering of G by induction. Specifically, let us claim by induction that every DAG has a topological ordering. This is clearly true for DAGs on one or two nodes. Now suppose it is true for DAGs with up to some number of nodes n . Then, given a DAG G on $n + 1$ nodes, we find a node v with no incoming edges, as guaranteed by (3.19). We place v first in the topological ordering; this is safe, since all edges out of v will point forward. Now $G - \{v\}$ is a DAG, since deleting v cannot create any cycles that weren't there previously. Also, $G - \{v\}$ has n nodes, so we can apply the induction hypothesis to obtain a topological ordering of $G - \{v\}$. We append the nodes of $G - \{v\}$ in this order after v ; this is an ordering of G in which all edges point forward, and hence it is a topological ordering.

Thus we have proved the desired converse of (3.18).

(3.20) *If G is a DAG, then G has a topological ordering.*

The inductive proof contains the following algorithm to compute a topological ordering of G .

```

To compute a topological ordering of  $G$ :
Find a node  $v$  with no incoming edges and order it first
Delete  $v$  from  $G$ 
Recursively compute a topological ordering of  $G - \{v\}$ 
    and append this order after  $v$ 

```

In Figure 3.8 we show the sequence of node deletions that occurs when this algorithm is applied to the graph in Figure 3.7. The shaded nodes in each iteration are those with no incoming edges; the crucial point, which is what

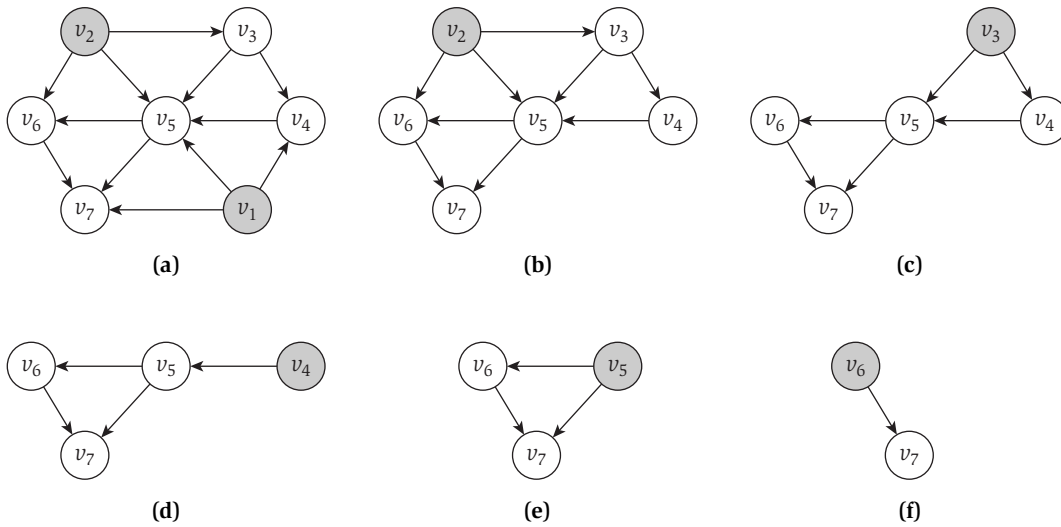


Figure 3.8 Starting from the graph in Figure 3.7, nodes are deleted one by one so as to be added to a topological ordering. The shaded nodes are those with no incoming edges; note that there is always at least one such edge at every stage of the algorithm's execution.

(3.19) guarantees, is that when we apply this algorithm to a DAG, there will always be at least one such node available to delete.

To bound the running time of this algorithm, we note that identifying a node v with no incoming edges, and deleting it from G , can be done in $O(n)$ time. Since the algorithm runs for n iterations, the total running time is $O(n^2)$.

This is not a bad running time; and if G is very dense, containing $\Theta(n^2)$ edges, then it is linear in the size of the input. But we may well want something better when the number of edges m is much less than n^2 . In such a case, a running time of $O(m + n)$ could be a significant improvement over $\Theta(n^2)$.

In fact, we can achieve a running time of $O(m + n)$ using the same high-level algorithm—iteratively deleting nodes with no incoming edges. We simply have to be more efficient in finding these nodes, and we do this as follows.

We declare a node to be “active” if it has not yet been deleted by the algorithm, and we explicitly maintain two things:

- (a) for each node w , the number of incoming edges that w has from active nodes; and
- (b) the set S of all active nodes in G that have no incoming edges from other active nodes.

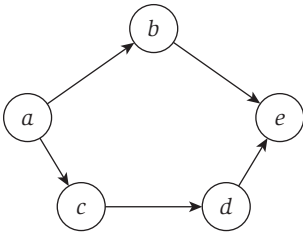


Figure 3.9 How many topological orderings does this graph have?

At the start, all nodes are active, so we can initialize (a) and (b) with a single pass through the nodes and edges. Then, each iteration consists of selecting a node v from the set S and deleting it. After deleting v , we go through all nodes w to which v had an edge, and subtract one from the number of active incoming edges that we are maintaining for w . If this causes the number of active incoming edges to w to drop to zero, then we add w to the set S . Proceeding in this way, we keep track of nodes that are eligible for deletion at all times, while spending constant work per edge over the course of the whole algorithm.

Solved Exercises

Solved Exercise 1

Consider the directed acyclic graph G in Figure 3.9. How many topological orderings does it have?

Solution Recall that a topological ordering of G is an ordering of the nodes as $v_1, v_2, \dots, v_n$ so that all edges point “forward”: for every edge (v_i, v_j) , we have $i < j$.

So one way to answer this question would be to write down all $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$ possible orderings and check whether each is a topological ordering. But this would take a while.

Instead, we think about this as follows. As we saw in the text (or reasoning directly from the definition), the first node in a topological ordering must be one that has no edge coming into it. Analogously, the last node must be one that has no edge leaving it. Thus, in every topological ordering of G , the node a must come first and the node e must come last.

Now we have to figure how the nodes b , c , and d can be arranged in the middle of the ordering. The edge (c, d) enforces the requirement that c must come before d ; but b can be placed anywhere relative to these two: before both, between c and d , or after both. This exhausts all the possibilities, and so we conclude that there are three possible topological orderings:

a, b, c, d, e

a, c, b, d, e

a, c, d, b, e

Solved Exercise 2

Some friends of yours are working on techniques for coordinating groups of mobile robots. Each robot has a radio transmitter that it uses to communicate

with a base station, and your friends find that if the robots get too close to one another, then there are problems with interference among the transmitters. So a natural problem arises: how to plan the motion of the robots in such a way that each robot gets to its intended destination, but in the process the robots don't come close enough together to cause interference problems.

We can model this problem abstractly as follows. Suppose that we have an undirected graph $G = (V, E)$, representing the floor plan of a building, and there are two robots initially located at nodes a and b in the graph. The robot at node a wants to travel to node c along a path in G , and the robot at node b wants to travel to node d . This is accomplished by means of a *schedule*: at each time step, the schedule specifies that one of the robots moves across a single edge, from one node to a neighboring node; at the end of the schedule, the robot from node a should be sitting on c , and the robot from b should be sitting on d .

A schedule is *interference-free* if there is no point at which the two robots occupy nodes that are at a distance $\leq r$ from one another in the graph, for a given parameter r . We'll assume that the two starting nodes a and b are at a distance greater than r , and so are the two ending nodes c and d .

Give a polynomial-time algorithm that decides whether there exists an interference-free schedule by which each robot can get to its destination.

Solution This is a problem of the following general flavor. We have a set of possible *configurations* for the robots, where we define a configuration to be a choice of location for each one. We are trying to get from a given starting configuration (a, b) to a given ending configuration (c, d) , subject to constraints on how we can move between configurations (we can only change one robot's location to a neighboring node), and also subject to constraints on which configurations are "legal."

This problem can be tricky to think about if we view things at the level of the underlying graph G : for a given configuration of the robots—that is, the current location of each one—it's not clear what rule we should be using to decide how to move one of the robots next. So instead we apply an idea that can be very useful for situations in which we're trying to perform this type of search. We observe that our problem looks a lot like a path-finding problem, not in the original graph G but in the space of all possible configurations.

Let us define the following (larger) graph H . The node set of H is the set of all possible configurations of the robots; that is, H consists of all possible pairs of nodes in G . We join two nodes of H by an edge if they represent configurations that could be consecutive in a schedule; that is, (u, v) and (u', v') will be joined by an edge in H if one of the pairs u, u' or v, v' are equal, and the other pair corresponds to an edge in G .

We can already observe that paths in H from (a, b) to (c, d) correspond to schedules for the robots: such a path consists precisely of a sequence of configurations in which, at each step, one robot crosses a single edge in G . However, we have not yet encoded the notion that the schedule should be interference-free.

To do this, we simply delete from H all nodes that correspond to configurations in which there would be interference. Thus we define H' to be the graph obtained from H by deleting all nodes (u, v) for which the distance between u and v in G is at most r .

The full algorithm is then as follows. We construct the graph H' , and then run the connectivity algorithm from the text to determine whether there is a path from (a, b) to (c, d) . The correctness of the algorithm follows from the fact that paths in H' correspond to schedules, and the nodes in H' correspond precisely to the configurations in which there is no interference.

Finally, we need to consider the running time. Let n denote the number of nodes in G , and m denote the number of edges in G . We'll analyze the running time by doing three things: (1) bounding the size of H' (which will in general be larger than G), (2) bounding the time it takes to construct H' , and (3) bounding the time it takes to search for a path from (a, b) to (c, d) in H .

1. First, then, let's consider the size of H' . H' has at most n^2 nodes, since its nodes correspond to pairs of nodes in G . Now, how many edges does H' have? A node (u, v) will have edges to (u', v) for each neighbor u' of u in G , and to (u, v') for each neighbor v' of v in G . A simple upper bound says that there can be at most n choices for (u', v) , and at most n choices for (u, v') , so there are at most $2n$ edges incident to each node of H' . Summing over the (at most) n^2 nodes of H' , we have $O(n^3)$ edges.

(We can actually give a better bound of $O(mn)$ on the number of edges in H' , by using the bound (3.9) we proved in Section 3.3 on the sum of the degrees in a graph. We'll leave this as a further exercise.)

2. Now we bound the time needed to construct H' . We first build H by enumerating all pairs of nodes in G in time $O(n^2)$, and constructing edges using the definition above in time $O(n)$ per node, for a total of $O(n^3)$. Now we need to figure out which nodes to delete from H so as to produce H' . We can do this as follows. For each node u in G , we run a breadth-first search from u and identify all nodes v within distance r of u . We list all these pairs (u, v) and delete them from H . Each breadth-first search in G takes time $O(m + n)$, and we're doing one from each node, so the total time for this part is $O(mn + n^2)$.